

Q1. Student Grade Book Application

CODE

```
def get_grade(avg):
    if avg >= 90:
        return 'O'
    elif avg >= 80:
        return 'A+'
    elif avg >= 70:
        return 'A'
    elif avg >= 60:
        return 'B+'
    elif avg >= 50:
        return 'B'
    else:
        return 'F'
student_data = {
    "Alice": [85, 92, 78, 90, 88],
    "Bob": [95, 98, 92, 94, 96],
    "Charlie": [65, 70, 58, 62, 68],
    "David": [45, 52, 48, 50, 42],
    "Eva": [88, 76, 82, 80, 85]
}
subject_names = ("Subject 1", "Subject 2", "Subject 3", "Subject 4", "Subject 5")
processed_data = {
    name: {
        "marks": marks,
        "total": sum(marks),
        "average": sum(marks) / len(marks),
        "grade": get_grade(sum(marks) / len(marks))
    }
    for name, marks in student_data.items()
}
topper = max(processed_data.items(), key=lambda item: item[1]["average"])
num_subjects = len(subject_names)
subject_means = [
    sum(marks[i] for marks in student_data.values()) / len(student_data)
    for i in range(num_subjects)
]
highest_subject_idx = subject_means.index(max(subject_means))
highest_subject_name = subject_names[highest_subject_idx]
highest_subject_score = subject_means[highest_subject_idx]
unique_grades = {info["grade"] for info in processed_data.values()}
sorted_students = sorted(
    processed_data.items(),
    key=lambda item: item[1]["average"]
)
print("=" * 65)
print(f'{ "Name":<12} | { "Marks":<22} | { "Total":<6} | { "Avg":<6} | { "Grade":<5}'}
print("=" * 65)
for name, info in sorted_students:
    marks_str = str(info["marks"])
    print(f'{name:<12} | {marks_str:<22} | {info["total"]:<6} | {info["average"]:<6.2f} | {info["grade"]:<5}')
print("=" * 65)
print(f'\nClass Topper: {topper[0]} with an Average of {topper[1]["average"]:.2f}')
print(f'Highest Performing Subject: {highest_subject_name} (Mean Score: {highest_subject_score:.2f})')
print(f'\nGrades Awarded Across Class: {', '.join(sorted(unique_grades))}')
```

OUTPUT

8/5/26, 11:20 AM

Student Submissions — Application Based Questions

SIMATS Engineering • CSE • CSA0804 — Python Programming • 05-08-2026 • Category: Application Based Questions • Student Sub

Q2. Debugging and Rewriting

CODE

```
accounts = {}
def create_account(acc_no, name, balance):
    accounts[acc_no] = {'name': name, 'balance': balance, 'txns': []}
def deposit(acc_no, amount):
    if acc_no in accounts:
        accounts[acc_no]['balance'] += amount
        accounts[acc_no]['txns'].append(('Deposit', amount))
def withdraw(acc_no, amount):
    if accounts[acc_no]['balance'] >= amount:
        accounts[acc_no]['balance'] -= amount
        accounts[acc_no]['txns'].append(('Withdraw', amount))
    else:
        print('Insufficient balance')
def mini_statement(acc_no):
    for t in accounts[acc_no]['txns'][-5:]:
        print(t)
create_account(1001, 'Ravi', 10000)
deposit(1001, 5000)
withdraw(1001, 20000)
withdraw(1001, 3000)
mini_statement(1001)
print('Balance:', accounts[1001]['balance'])
```